

Arrays

CSC100 Introduction to programming in C/C++ - Spring 2026

Table of Contents

- [Overview](#)
- [Collections in other languages](#)
- [What is an array?](#)
- [One-dimensional arrays](#)
- [Declaring arrays](#)
- [Declaring Arrays in C](#)
- [Reading array values](#)
- [Default values and garbage data](#)
- [Fixing initialization](#)
- [Character array experiments](#)
- [PRACTICE Writing array declarations \(15 min\)](#)
- [Array length](#)
- [Iterating over arrays](#)
- [Iteration examples](#)
- [Initialization with Pattern](#)
- [PRACTICE Input and Count](#)

Overview

- Variables that can hold only a single data item (a number or a character, which is a number, too) are called **scalars**: 1, 'a'
- In mathematics, **ordered tuples** of data ($x_{\{1\}}, \dots x_{\{n\}}$) are called **vectors**. In the R code below, a vector *v* is defined and printed:

```
c(1,2,3) -> v ## create a vector of three numbers  
v
```

```
[1] 1 2 3
```

- In C there are two **aggregate** structures that can store *collections* of values: **arrays** and **structures**.
- A **structure** is a forerunner of a **class**, a concept that becomes central in C++, which is also called "C with classes".
- Classes contain objects and their properties, and they are a core concept for **object-oriented programming** (OOP).

Collections in other languages

- Different programming languages have different data structures. The language Python has **dictionaries**, the language R has **data frames**, and the language Lisp has **lists**¹:
- Example with **Python**: a *dictionary* of car data.

```
thisDict = {
    "brand": "Ford",      # key: brand attribute, value: Ford
    "model": "Mustang",  # key: model attribute, value: Mustang
    "year": 1964         # key: year attribute, value: 1964
}
for key, value in thisDict.items(): # items are encapsulated
    print(f"key: {key}, value: {value}")
```

```
key: brand, value: Ford
key: model, value: Mustang
key: year, value: 1964
```

- Example with **R**: a *data frame* of tooth growth data, consisting of three different vectors of the same length but different data types.

```
str(ToothGrowth) # prints the structure of the data frame
```

```
'data.frame':      60 obs. of  3 variables:
 $ len : num  4.2 11.5 7.3 5.8 6.4 10 11.2 11.2 5.2 7 ...
 $ supp: Factor w/ 2 levels "OJ","VC": 2 2 2 2 2 2 2 2 2 2 ...
 $ dose: num  0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 ...
```

```
head(ToothGrowth, 5)
```

```
   len supp dose
1  4.2   VC  0.5
2 11.5   VC  0.5
3  7.3   VC  0.5
4  5.8   VC  0.5
5  6.4   VC  0.5
```

- For **Lisp**, the **list** is the fundamental data structure:

```
(setq my-list '(1 2 3 4 5))
(message "List contents: %s" my-list)
```

What is an array?

- An **array** is a *data structure* containing a number of data values, all of which have the same type (like int, char or float).
- You can visualize arrays as sorted box collections.

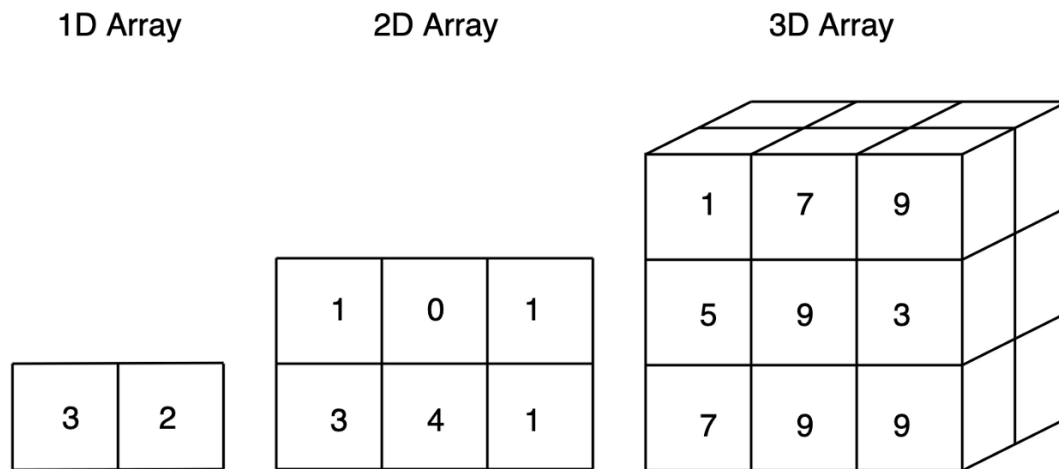


Figure 1: Arrays of different dimensions with values in them

- The computer stores them differently - sequentially as a set of memory addresses. Example: 2D matrix of characters, stored in row-major fashion:

`char sports[5][8]` (row-major layout)

g 1000	o 1001	l 1002	f 1003	\0 1004	 1005	 1006	 1007
h 1008	o 1009	c 1010	k 1011	e 1012	y 1013	\0 1014	 1015
f 1016	o 1017	o 1018	t 1019	b 1020	a 1021	l 1022	\0 1023
c 1024	r 1025	i 1026	c 1027	k 1028	e 1029	t 1030	\0 1031
s 1032	h 1033	o 1034	o 1035	t 1036	i 1037	n 1038	g 1039

One-dimensional arrays

- The simplest kind of array has one dimension - conceptually arranged visually in a single row (or column).

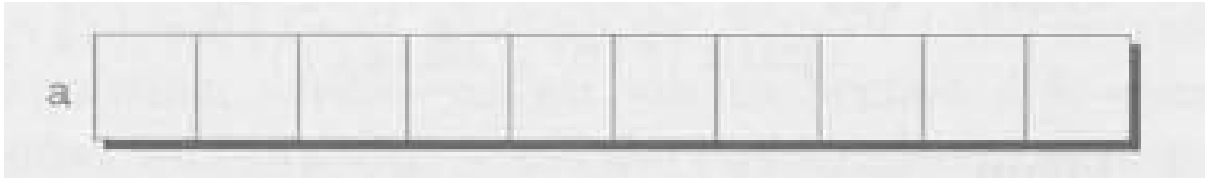


Figure 2: Visualization of a 1-dim array 'a' (Source: King)

- Each element of an array of type T is treated as if it were a variable of type T. Here are three short examples:

```
for ( int i = 0; i < N; i++ )
    a[i] = 0;                /* clears a */

for ( int i = 0; i < N; i++ )
    scanf("%d", &a[i]);      /* reads data into a */

for ( int i = 0; i < N; i++ )
    sum += a[i];             /* sums the elements of a */
                             /* sum += a[i] => sum = sum + a[i] */
```

Declaring arrays

- To declare an array, we must specify the *type* and *number* of its elements, e.g. for an array of 10 elements:

```
int a[10]; // type T + name + dimension
printf("a[1] = %d\n a[9] = %d\n", a[1], a[9]);
```

```
a[1] = 0
a[9] = 0
```

- The array must be initialized, just like any scalar variable, to be of use to us (otherwise strange values may appear):

```
int a[10]; // the values are undefined right now
for (int i=0; i<10; i++) printf("%d ", a[i]);
```

```
2 0 -1075053569 0 2053733817 32765 100 0 4096 0
```

- You can initialize arrays explicitly using {...}:

```
int int_array[5] = {10,20,30,40,50};
double double_array[] = {2.1,2.2,2.3,2.4,2.5};
char char_array[] = {'h','e','l','l','o','\0'};
```

- This is how `char_array` looks like (the last character `\0` is only a terminating character):

'H'	'e'	'l'	'l'	'o'
0	1	2	3	4

- Control over start/finish of arrays is essential, otherwise you incur a so-called **memory overflow**:

```
char c1[] = {'H','e','l','l','o','\0'}; // correct
char c2[] = {'h','e','l','l','o'}; // incorrect
printf("%s\n%s",c1,c2);
```

```
Hello
helloHello
```

Declaring Arrays in C

Code along with me in OneCompiler.com/c if you like.

Reading array values

- **Question:** What values do you expect this program to print? Explain the output.

```
int arr[4] = {10, 20};
printf("arr[0] = %d\n", arr[0]);
printf("arr[1] = %d\n", arr[1]);
printf("arr[2] = %d\n", arr[2]);
printf("arr[3] = %d\n", arr[3]);
```

```
arr[0] = 10
arr[1] = 20
arr[2] = 0
arr[3] = 0
```

- **Explanation**

Only the first two elements are initialized, the rest default to zero.

Default values and garbage data

- **Question:** What happens when you declare an array without initializing it? What values do you see and why?

```
int nums[6] = {5,10,'c'}; // define integer array of 6 values
for(int i=0 ; i<6 ; i++ )
    printf("%d ", nums[i]); // values are undefined
```

```
5 10 99 0 0 0
```

- **Explanation:**

Uninitialized local arrays contain garbage values - though your computer may show them as '0'.

Fixing initialization

- **Question:** Update the previous program and initialize the array in two different ways.

Initialize array values with a loop

- Initialize the array nums to the value 1:

Initialize array with an initializer list

- Initialize the array nums to the value 1 using an initializer list.

Character array experiments

- Consider this code:

- **Question:** What will this code print? Why does word1 behave differently from word2?

- **Explanation**

- Strings like "hello" are stored as arrays. This is how you will do it later:

```
char *word = "hello"; // `word` is a `char` pointer to `h`  
printf("word: %s\n", word); // prints the string  
  
for (int p=0; p < 5 ; p++) { // pointer arithmetic  
    printf("word: %c\n", word[p]);  
}
```

```
word: hello  
word: h  
word: e  
word: l  
word: l  
word: o
```

PRACTICE Writing array declarations (15 min)

- **Task:** Write C declarations for the following array scenarios, then print them. Starter code: tinyurl.com/array-declarations
 1. An array a of 10 integers.
 2. An array b of 5 floats initialized to 1.1, 2.2,...,5.5
 3. A character array c initialized to the word "Hi"
 4. An array d of 100 doubles initialized to 0 (print first and last five elements only)
 5. An array e of 2 Boolean values whose conjunction (&) is TRUE.
- Sample output

```
0 0 0 0 0 0 0 0 0 0
1.1 2.2 3.3 4.4 5.5
H i
0 0 0 0 0 0 0 0 0 0
1 AND 1 is 1 (TRUE)
```

- Submit URL: **In-class practice 10: writing array declarations.**

Solution:

```
// 1. An array of 10 integers
int a[10];
for (int i = 0; i < 10; i++) printf("%d ", a[i]);
// 2. An array of 5 floats initialized to 1.1, 2.2, ..., 5.5
float b[] = {1.1, 2.2, 3.3, 4.4, 5.5}; puts("");
for (int i = 0; i < 5; i++) printf("%.1f ", b[i]); puts("");
// 3. A character array initialized to the word "Hi"
char c[] = {'H', 'i', '\0'};
for (int i = 0; i < 2; i++) printf("%c ", c[i]); puts("");
// 4. An array of 100 doubles initialized to 0
double d[100] = {0};
// PRINT the first and the last five elements only
for (int i = 0; i < 100; i++) {
    if (i < 5 || i > 94)
        printf("%g ", d[i]);
}
// 5. An array e of 2 Boolean values whose conjunction (&) is TRUE
int e[2] = {1,1};
printf("\n%d AND %d is %d (TRUE)\n", e[0], e[1], e[0] & e[1]);
```

```
1 0 0 0 651413768 31812 651523680 31812 -2080208528 32767
1.1 2.2 3.3 4.4 5.5
H i
0 0 0 0 0 0 0 0 0 0
1 AND 1 is 1 (TRUE)
```

Array length

- An array can have any length (as long as there's memory available). Since the length may have to be adjusted, it can be useful to define it as a macro with `#define`.

```
#define N 10 // substitute 10 for N everywhere in the program
int a[N]; // declare array of 10 elements indexed from 0 to 9
printf("%d ", a[N]); // prints a[10] which is undefined
```

808220928

- Remember that now N will **blindly** be replaced by 10 **everywhere** in the program by the pre-processor.

Iterating over arrays

- for loops are made for arrays. Here are a few examples. Can you see what each of them does?

```
for (i = 0; i < 10 ; i++ )  
    a[i] = 0; // or you could have declared `int a[10] = {0};`
```

0 is assigned to a[0] to a[9]

```
for (i = 0; i < 10 ; i++ )  
    scanf("%d", &a[i]);
```

Assigns integer input (keyboard) to a[0] through a[9]

```
for (i = 0; i < 10 ; i++ )  
    sum += a[i];
```

sum <- sum + a[0] + a[1] + ... + a[10]

Iteration examples

- These short problems build on the three examples you've just seen.
- Open a new file at OneCompiler.Com and put all of them into it.
- Pseudocode first!

Initialization with Pattern

- Initialize the array a with the values 1,2, ..., 10 using a for loop that starts at i = 0;

```
// SET array of 10 elements  
int a[10];  
// FOR i from 0 to 10: DO  
for (int i=0; i<10; i++) {  
    // initialize the array elements  
    a[i] = i + 1;  
    // print array element  
    printf("%d ", a[i]);  
} // END FOR
```

1 2 3 4 5 6 7 8 9 10

PRACTICE Input and Count

- Read 5 whole (non-negative integer) numbers into an array b and count how many of them are even:

```
// SET array b of 5 elements
int b[5];
// SET counter to 0
int count = 0;
// PRINT "Enter 5 non-negative integer numbers:"
printf("Enter 5 non-negative integer numbers: ");
// FOR i from 0 to 10: DO
for (int i = 0; i < 5; i++) {
    // READ array element
    scanf("%d", &b[i]);
    // PRINT array element
    printf("%d ", b[i]);
    // IF array element is even
    if (b[i] % 2 == 0) { // even if remainder of b[i]/2 is zero
        // ADD 1 to counter
        count++;
    } // END IF
} // END FOR
printf("\nNumber of even values is %d\n", count);
```

```
Enter 5 non-negative integer numbers: 10 21 33 4 5
Number of even values is 2
```

- How could this be generalized?
 1. Accept arrays of any length.
 2. Abort gracefully when the entry is not a whole number
- Input:

```
echo 10 21 33 4 5 > input
cat input
```

```
10 21 33 4 5
```

Footnotes:

¹ **Emacs** (my editor) is programmed in Lisp, which is also the oldest language for AI applications; **Python** is an important general purpose language which dominates machine learning; and **R** is a language for statistics and data visualization. If you study computer or data science at Lyon, you will learn all of these languages.

Author: Marcus Birkenkrahe

Created: 2026-04-12 Sun 12:00